

TRABAJO FINAL DE GRADO

CONCEPTO DE RÚBRICA

Implementación de IA Agéntica para la Creación de Rúbricas

Un caso de estudio aplicado a la materia Organización Empresarial

INTEGRANTES

Renzo Sosa · Emmanuel Avellaneda

Organización Empresarial

Profesor: Alberto Cortez

2026

Índice

1. Introducción

2. Marco teórico

¿Qué es una rúbrica de evaluación?

¿Qué es una IA agéntica?

¿Qué es una skill?

3. Instalación y configuración del entorno

Instalación de la IA agéntica (Claude Code)

Instalación de la skill `rubrica-builder`

4. Normalización de la consigna y de la entrega

5. Creación de la rúbrica (modo CREAR)

6. Auditoría de la rúbrica (modo AUDITAR)

7. Validación estructural de la rúbrica

8. Testeo de la rúbrica (modo TEST)

9. Ajuste iterativo de consigna, rúbrica y entrega

10. Caso de estudio aplicado: Organización Empresarial

11. Conclusiones finales

12. Fuentes utilizadas y reconocimiento de autoría

1. Introducción

El presente trabajo tiene como objetivo documentar la implementación de una **IA agéntica** —en este caso, **Claude Code**— junto con una extensión de conocimiento específica denominada **skill**, para la creación, curado, auditoría y testeo de **rúbricas de evaluación autosuficientes**, en el marco del sistema de corrección automática **Active-IA**.

A diferencia de un uso conversacional simple de un modelo de lenguaje (preguntar y recibir una respuesta), una IA agéntica es capaz de **ejecutar un proceso de varios pasos, tomar decisiones intermedias, usar herramientas** (scripts, archivos, terminal) y **validar sus propios resultados** antes de entregarlos. Este trabajo toma como skill de referencia `rubrica-builder`, desarrollada por Juan Cruz Robledo, y la utiliza como caso de aplicación práctica para transformar la consigna de un Trabajo Práctico en una rúbrica de evaluación estructurada, coherente y —sobre todo— **corregible por otra IA sin ambigüedad**.

Como caso de estudio transversal a lo largo de todo el informe se utilizan las rúbricas reales elaboradas para la materia **Organización Empresarial (OE)**, ya que es el contexto en el que se origina esta tesis.

Video de la tesis (fundamentos y motivaciones):
https://www.youtube.com/watch?v=OOzTbtUt_-Q

2. Marco teórico

2.1. ¿Qué es una rúbrica de evaluación?

Una rúbrica es un instrumento de evaluación que descompone un trabajo práctico en **criterios** (grandes bloques evaluables, cada uno con un peso o ponderación) y **subcriterios** (verificaciones puntuales dentro de un criterio, sin nota propia). Cada subcriterio se apoya en **evidencias**: afirmaciones verificables de forma binaria (cumple / no cumple) mirando únicamente la entrega del alumno.

*Contenido ampliado en el video "¿Qué es una rúbrica?", disponible en:
drive.google.com/drive/folders/1WewN_uwqNbvnyhkBqQ4WZ2_lge4UHPCU
(referenciado también en la sección de fuentes).*

El punto central que este trabajo busca remarcar —y que constituye el eje conceptual de toda la skill— es que una rúbrica pensada para ser corregida por una IA debe ser **autosuficiente**: en el momento de la corrección automática, el modelo que corrige **no vuelve a leer la consigna original**. Solo tiene ante sí dos elementos: la rúbrica y la entrega. En consecuencia, todo requisito de la consigna que no haya quedado plasmado explícitamente en la rúbrica **directamente no se evalúa**, sin importar cuán importante fuera para el docente.

De esta premisa se deriva la segunda regla estructural que gobierna el diseño de una rúbrica: **el puntaje vive en el criterio, no en el subcriterio**. Si un requisito necesita poder penalizarse de forma independiente, debe modelarse como **criterio propio** con su peso, y no quedar diluido dentro de un criterio más amplio. De lo contrario, el corrector detecta la falla pero no tiene "de dónde restar" sin perjudicar a otros requisitos agrupados en el mismo criterio, lo cual vuelve la corrección **no determinista** (la misma entrega podría obtener notas distintas según el modelo o la corrida).

2.2. ¿Qué es una IA agéntica?

Una IA agéntica es un sistema basado en un modelo de lenguaje que, a diferencia de un chatbot tradicional, tiene la capacidad de:

- Interpretar un objetivo complejo y **planificar los pasos** necesarios para alcanzarlo.
- **Usar herramientas** del entorno en el que corre (leer y escribir archivos, ejecutar scripts, correr comandos de terminal, consultar documentación).
- **Verificar su propio trabajo** antes de darlo por terminado (por ejemplo, corriendo un script de validación y corrigiendo el resultado si falla).
- Operar en **múltiples turnos**, sosteniendo contexto y tomando decisiones intermedias sin que el usuario deba especificar cada micro-paso.

En este trabajo la IA agéntica utilizada es **Claude Code**, de Anthropic, un agente de línea de comandos pensado originalmente para tareas de programación, pero cuya arquitectura de "modelo + herramientas + instrucciones" es perfectamente aplicable a un dominio no estrictamente técnico como la elaboración de instrumentos de evaluación.

El agente, por sí solo, no "sabe" cómo se arma una rúbrica válida para el sistema Active-IA. Ese conocimiento específico del dominio se le agrega a través de una **skill**, concepto que se desarrolla a continuación.

2.3. ¿Qué es una skill?

Una skill es un **paquete de conocimiento especializado** que se instala sobre la IA agéntica para extender sus capacidades a un dominio concreto. Técnicamente se compone de:

- Un archivo `SKILL.md`, con una descripción (usada por el agente para saber *cuándo* activarse) y las instrucciones detalladas de *cómo* ejecutar la tarea.
- Archivos de referencia (`references/`) con el conocimiento de dominio que el agente debe respetar como fuente de verdad (por ejemplo, el esquema exacto que debe tener el JSON de una rúbrica, o los límites técnicos de lo que el corrector automático puede o no puede leer).
- Scripts auxiliares (`scripts/`) que el propio agente ejecuta para **validar** o **simular** su trabajo, en lugar de confiar únicamente en su propio criterio.
- Ejemplos (`assets/` , `examples/`) que sirven de molde.

La skill utilizada en este trabajo, `rubrica-builder`, define **cuatro modos de funcionamiento** (CURAR, CREAR, AUDITAR y TEST), que se corresponden,

respectivamente, con las secciones 4, 5, 6 y 8 de este informe.

3. Instalación y configuración del entorno

3.1. Instalación de la IA agéntica (Claude Code)

Claude Code se instala como una herramienta de línea de comandos (CLI). Los pasos generales son:

1. Contar con **Node.js** instalado (requisito de la herramienta de instalación).
2. Instalar el CLI mediante el gestor de paquetes `npm`.
3. Autenticarse con una cuenta de Anthropic (API key o suscripción, según el plan).
4. Verificar la instalación ejecutando el comando del agente dentro de una carpeta de trabajo (en este caso, la carpeta del repositorio de la materia Organización Empresarial).

3.2. Instalación de la skill `rubrica-builder`

La skill se instala dentro del entorno del agente mediante el comando indicado en su propia documentación:

```
npx skills add https://github.com/JuanCruzRobledo/rubrica-builder
```

Este comando descarga la skill y la deja disponible para que el agente la cargue **automáticamente** cuando detecta, por el pedido del usuario, que la tarea corresponde a alguno de sus cuatro modos (por ejemplo: *"armá los criterios para este TP"*, *"revisá si esta rúbrica cubre todo lo que pide el parcial"*, *"probá cómo corrige esta rúbrica con esta entrega"*). No es necesario invocarla por nombre explícitamente: el agente la reconoce por el contexto del pedido.

Requisitos técnicos declarados por la skill:

- Python 3.10 o superior (usa únicamente librería estándar, sin dependencias externas) — necesario para los scripts de validación y testeo.
- El CLI de Claude Code disponible en el `PATH` del sistema — necesario específicamente para el modo TEST, que dispara una corrección real con el modelo.

4. Normalización de la consigna y de la entrega

Este es, en la práctica, el paso más subestimado y el que más impacto tiene sobre la calidad final de la rúbrica. Antes de generar cualquier criterio, hay que preguntarse: **¿esta consigna, tal como está redactada, se puede corregir con una IA?**

El motivo es que el corrector automático de Active-IA tiene **límites duros** que no dependen de qué tan bien esté escrita la rúbrica:

1. **No navega a recursos externos.** Un link a un repositorio de GitHub, una URL de un deploy o un video en YouTube/Drive nunca se abren: son invisibles para el corrector.
2. **Filtra archivos por extensión según el modo de corrección** (`solo_codigo` , `web_completo` , `proyecto_completo` o `personalizado`). Las imágenes (`.png` , `.jpg` , `.jpeg` , `.gif` , etc.) están explícitamente excluidas en los modos de código: **nunca llegan al corrector**.
3. **Es un solo flujo: código O informe en PDF, nunca ambos** en la misma corrección.
4. **No ejecuta nada.** El corrector lee archivos estáticos; no corre el programa, no verifica que un deploy responda, no corre una batería de tests.

De este diagnóstico surge el **modo CURAR** de la skill, cuya función es analizar la consigna original y, por cada punto que rompa alguno de los cuatro límites anteriores, proponer uno de tres destinos:

Destino	Cuándo se aplica	Ejemplo
Adaptar (preferido)	El requisito se puede reformular para que entre en el flujo único, sin perder su intención pedagógica	"Adjuntá el link del repo" → "Entregá un .zip del proyecto con el README mostrando la URL del repo"
Mantener + corrección manual	El requisito es importante pero no admite adaptación (ej. un video de defensa oral)	Se deja igual en la consigna, se marca como corrección manual, y no entra en la rúbrica
Sacar (solo de la rúbrica)	El punto es accesorio para la nota (un link decorativo)	No se escribe como criterio; la consigna lo conserva igual

Un principio importante que distingue este modo del resto: **la consigna nunca pierde contenido** por decisión unilateral del agente. Lo que se "saca" es exclusivamente de la rúbrica, mientras que la consigna se adapta o se conserva, y el proceso es siempre conversacional: el agente propone una cura concreta y pregunta antes de aplicarla.

Este es, precisamente, el criterio pedagógico que se aplicó al confeccionar las rúbricas de Organización Empresarial que se documentan en la sección 10: al ser una materia no estrictamente de programación, se decidió de antemano que todas las entregas de OE se resuelven en **texto plano**, evitando de raíz pedir capturas de pantalla, diagramas en imagen o links externos. Cuando la consigna original requería representar un esquema, se optó por describirlo en texto o, en materias con contenido técnico afín, expresarlo en formatos de texto estructurado (por ejemplo, notación Mermaid o XML) en lugar de una imagen.

4.1. Caso de estudio: estandarizar la tecnología antes de armar la rúbrica (evaluación de control de versiones con Git)

El paso de normalización no se agota en el formato del archivo de entrega: también implica decidir, **antes** de escribir un solo criterio, qué tecnología concreta va a usar toda la cursada para resolver una consigna, de manera que exista un único formato de evidencia posible para auditar. El siguiente caso, documentado en video (ver Fuentes, §12), ilustra este razonamiento aplicado a un TP cuyo objetivo es evaluar el uso de control de versiones.

El punto de partida es que el TP ideal para un corrector automático es aquel que se resuelve enteramente con un PDF de preguntas y respuestas: es el formato más fácil de auditar, porque el corrector solo necesita leer texto. Pero no todos los temas de la cursada se prestan a ese formato. Un TP sobre control de versiones, por definición, no puede evaluarse solo con teoría: hay que verificar que el alumno efectivamente creó ramas, hizo commits, aplicó un merge. Ahí aparece la primera decisión de normalización: **unificar la tecnología**. Se optó por Git como sistema de control de versiones para toda la cursada, de modo que los alumnos entregaran evidencia comparable entre sí en lugar de un mosaico de herramientas distintas.

La segunda decisión fue el formato de esa evidencia. La opción más intuitiva —pedir capturas de pantalla de la terminal mostrando cada comando ejecutado— se descartó deliberadamente: como ya se estableció en el diagnóstico de límites del corrector (§4),

las imágenes quedan excluidas en los modos de código y, aunque no lo estuvieran, una IA no es un método confiable para verificar a partir de una captura que un commit, una rama o un merge realmente existen en el historial del repositorio. Una captura se puede editar, recortar o simular; un repositorio Git no miente.

Consultando la documentación oficial de GitHub sobre la estructura interna de un repositorio, se identificó que la carpeta oculta `.git` —generada automáticamente por `git init`— contiene la totalidad de la información verificable de forma objetiva: el registro de commits, las referencias a las ramas (`refs/heads/`) y la configuración de identidad local. Es, en los hechos, el equivalente a un log de auditoría que no depende de lo que el alumno afirme haber hecho, sino de lo que el propio sistema de control de versiones registró.

La consigna original del TP1 (que pedía un PDF de teoría más capturas de pantalla de la Parte 2) se modificó en consecuencia: la entrega pasó a ser un único `.zip` que contiene el PDF con la resolución de la parte teórica **y la carpeta `.git`** del repositorio local, tal como se documenta en la consigna vigente del TP1 (§10) y se verifica en el criterio C1 de su rúbrica. Este caso resume el principio general de esta sección: antes de escribir un criterio hay que analizar qué es lo que realmente se necesita evaluar, reconocer las limitaciones concretas de la IA correctora, y recién entonces decidir qué tecnología y qué formato de entrega van a usar todos los alumnos por igual.

5. Creación de la rúbrica (modo CREAR)

Con la consigna ya normalizada (curada), el modo CREAR traduce el texto en una rúbrica completa. El proceso que ejecuta el agente es:

1. **Extracción de requisitos.** Lectura completa de la consigna, listando todo lo evaluable: funcionalidades o actividades pedidas, restricciones, condiciones de entrega, estructura obligatoria y prohibiciones. Todo requisito explícito debe terminar reflejado en algún criterio o evidencia; lo que no está en la consigna, no entra en la rúbrica.
2. **Agrupación en criterios según la "palanca de descuento".** No hay un número fijo de criterios: la cantidad la determina la cantidad de aspectos que el docente necesita poder penalizar de forma independiente. El **test de granularidad** es simple: *¿este requisito necesita bajar la nota por sí solo, sin afectar a otros?* → *criterio propio*. *¿Es solo una evidencia de verificación de algo más grande?* → *subcriterio*. A cada criterio se le asigna un peso proporcional a la importancia que la consigna le otorga, y la suma de todos los pesos debe dar exactamente **100**.
3. **Subcriterios con evidencias verificables.** Cada subcriterio se apoya en evidencias redactadas como afirmaciones binarias y objetivas, nunca subjetivas.
4. **Penalizaciones y condiciones de desaprobación,** solo si la consigna las establece explícitamente.
5. **Validación obligatoria** (ver sección 7) antes de dar la rúbrica por terminada.

Ejemplo aplicado (TP2 de Organización Empresarial)

Como ejemplo concreto, se tomó la consigna del **TP2 — Ingeniería de Prompts y Pensamiento Computacional**, que pide al alumno resolver 6 actividades relacionadas con técnicas de diseño de instrucciones para modelos de lenguaje. La rúbrica resultante (TPs/TP2/rubrica-tp2-ingenieria-prompts.json) quedó estructurada en 6 criterios —uno por actividad—, cada uno con su propio peso y sus `instrucciones_puntuacion` explícitas. A modo ilustrativo, el primer criterio quedó modelado así:

```
{  
  "id": "C1",
```

```
"nombre": "Actividad 1: Arquitectura de la Instrucción",
"peso": 18,
"instrucciones_puntuacion": "Si alguno de los 4 elementos fundamentales
está ausente, techo de 60% (10.8 pts). Si no hay delimitadores
visibles entre secciones, descontar 25% (4.5 pts)...",
"subcriterios": [
  { "id": "C1.1", "descripcion": "Los 4 elementos fundamentales están
  presentes...", "evidencias": ["...", "...", "...", "..."] },
  { "id": "C1.2", "descripcion": "El prompt usa delimitadores
  visibles...", "evidencias": ["..."] },
  { "id": "C1.3", "descripcion": "El prompt está orientado al caso
  pedido...", "evidencias": ["...", "..."] }
]
```

Nótese cómo cada subcriterio (C1.1, C1.2, C1.3) es un checklist de evidencias sin nota propia, mientras que la nota completa de la actividad depende del peso (18 puntos) del criterio C1, ajustado hacia abajo mediante reglas numéricas explícitas de descuento — exactamente el principio de granularidad descrito en la sección 2.1.

6. Auditoría de la rúbrica (modo AUDITAR)

Este modo no figuraba en el esquema orientativo inicial de este informe, pero se incorpora porque es uno de los **cuatro modos centrales** de la skill y cumple un rol de control de calidad que ninguno de los otros pasos cubre: revisar una rúbrica ya redactada (propia o de un tercero) contra la consigna original, con mirada de auditor.

El agente revisa, en este orden:

1. **Contradicciones** entre lo que pide la consigna y lo que premia o exige la rúbrica.
2. **Omisiones**: requisitos explícitos del TP que la rúbrica no evalúa. Es la falla más grave, porque lo omitido nunca se corrige.
3. **Requisitos diluidos**: puntos importantes que sí están en la rúbrica pero enterrados como subcriterio sin peso propio.
4. **Inversiones**: criterios o concesiones que la rúbrica agrega pero que la consigna no respalda en ningún lado.
5. **Elementos visuales** de la consigna que no se tradujeron a texto dentro de la rúbrica.
6. **Reajuste de pesos** a 100, si se agregaron o quitaron criterios.

El resultado de este modo es un breve resumen de hallazgos (qué se corrigió y por qué) seguido de la rúbrica corregida, lista para volver a validar.

7. Validación estructural de la rúbrica

Independientemente del modo utilizado, ninguna rúbrica se entrega sin pasar antes por el script `scripts/validar_rubrica.py`, incluido en la skill:

```
python scripts/validar_rubrica.py <ruta_al_json>
```

Este script replica en local las reglas que aplica el backend real de Active-IA: suma de pesos igual a 100, IDs con el patrón correcto, ausencia de evidencias vacías, nombres de campo correctos (por ejemplo, `nota_maxima` y no `nota_final`, un error presente incluso en cierta documentación desactualizada del propio sistema). Si el script falla, indica exactamente qué corregir; si pasa, imprime la confirmación "  **RÚBRICA VÁLIDA**", que se adjunta como evidencia en el informe final de cada rúbrica entregada.

Este paso es barato de ejecutar y evita el rechazo de la rúbrica al cargarla en el sistema real.

8. Testeo de la rúbrica (modo TEST)

Que una rúbrica sea **estructuralmente válida** no garantiza que **corrija bien**. El modo TEST cierra esa brecha: simula, en el entorno local, el pipeline real de corrección (consolidación de la entrega + armado del prompt de corrección), disparando la evaluación con el CLI de Claude en lugar del flujo de producción (N8N → Gemini). Los resultados son indicativos, pero permiten detectar problemas de diseño de la rúbrica antes de subirla al sistema real.

```
python scripts/simular_correccion.py \
  --rubrica <rubrica.json> \
  --entrega <archivo|carpeta|.zip|.txt> \
  --materia "Organización Empresarial" \
  --alumno "Alumno de prueba" \
  --modo solo_codigo
```

Para cada rúbrica se generan entregas **sintéticas** diseñadas a propósito para fallar en un único criterio a la vez (`s1_falla_c1` , `s2_falla_c2` , etc.), además de una entrega perfecta (`s0_perfecta`) y, en algunos casos, entregas con cumplimiento parcial. La lógica es la de un test unitario aplicado a una rúbrica: si el criterio C_n está bien diseñado, la entrega que falla exactamente en C_n debe producir una nota visiblemente más baja que la entrega perfecta, y el feedback generado debe señalar puntualmente esa falla.

Resultados obtenidos (TP2 — Organización Empresarial)

Entrega sintética	Qué testea	Nota esperada	Nota obtenida	Resultado
s0_perfecta	Rúbrica completa	≥ 90	100	✓
s1_falla_c1	C1 discrimina (18 pts)	~82	91.9	✓
s2_falla_c2	C2 discrimina (18 pts)	~82	82	✓
s3_falla_c3	C3 discrimina (18 pts)	~82	89.2	✓
s4_falla_c4	C4 discrimina (16 pts)	~84	47.0	✓

s5_falla_c5	C5 discrimina (16 pts)	~84	87	✓
s6_falla_c6	C6 discrimina (14 pts)	~86	87.5	✓
s7_parcial_c1	C1 con cumplimiento parcial	~91	88.3	✓

Fuente: `TPs/TP2/sinteticos/2026-06-22_175052/resumen.json`, generado por el script de testeo de la skill sobre la rúbrica del TP2.

Se observa que, en la mayoría de los criterios, la nota obtenida discrimina correctamente al alumno con la falla simulada respecto de la entrega perfecta. El caso de `s4_falla_c4` (nota 47.0 frente a un esperado de ~84) es un ejemplo de **criterio troncal**: al fallar, activa una condición de desaprobación o un techo de puntuación severo definido en `instrucciones_puntuacion`, lo cual es el comportamiento deseado cuando ese requisito es central para el trabajo. Este tipo de desvíos es justamente lo que motiva el siguiente paso: el ajuste iterativo.

Resultados obtenidos (TP1 — Primeros pasos con Git)

Entrega sintética	Qué testea	Nota esperada	Nota obtenida	Resultado
so_perfecta	Rúbrica completa	100	100	✓
s1_falla_c2	C2 discrimina (10 pts) — pregunta teórica sin responder	90	90	✓
s2_sin_git_cd2	CD2 discrimina — ausencia de la carpeta <code>.git</code>	≤30 (techo)	30	✓

Fuente: `TPs/TP1/sinteticos/2026-07-05_223422/`.

El caso `s2_sin_git_cd2` confirma el diseño intencional descrito en §4.1: la carpeta `.git` es el artefacto que respalda toda la parte práctica (C6-C9). Sin ella, esos cuatro criterios no tienen evidencia verificable (quedan en 0) y, adicionalmente, la condición de desaprobación `CD2` impone un techo de 30 puntos aunque la parte teórica (C2-C5) esté perfecta. El subtotal bruto antes del techo fue 42/100, lo que evidencia una redundancia de diseño: la ausencia de `.git` ya se penaliza dentro de C1 (−8 puntos) y además dispara el techo global de CD2, que termina siendo el factor decisivo.

Resultados obtenidos (TP5 — Live Coding)

Entrega sintética	Qué testea	Nota esperada	Nota obtenida	Resultado
so_perfecta	Rúbrica completa	100	100	✓
s1_falla_c4	C4 discrimina (18 pts) — define Boilerplate sin mencionar Snippets	91	91	✓
s2_zip_invalido_cd1	CD1 discrimina — entrega en <code>.rar</code> en vez de <code>.zip</code>	0 (techo)	0	✓

Fuente: [TPs/TP5/sinteticos/2026-07-05_223422/](#).

El caso `s2_zip_invalido_cd1` es el ejemplo más contundente de una condición de desaprobación: aunque el contenido teórico interno era correcto y completo en las cinco consignas, el sólo hecho de comprimir en `.rar` en vez de `.zip` —algo que la consigna prohíbe explícitamente— anula la nota por completo. Es la penalización más severa observada en todo el testeo, y la más evitable: un error de empaquetado, no de contenido.

Resultados obtenidos (TP6 — Automatización con n8n)

Entrega sintética	Qué testea	Nota esperada	Nota obtenida	Resultado
so_perfecta	Rúbrica completa	100	100	✓
s1_falla_c7	C7 discrimina (15 pts) — nombre de campo incorrecto en Edit Fields	~95 (solo C7)	91	✓
s2_sin_json_cd2	CD2 discrimina — ausencia del <code>.json</code> exportado de n8n	≤30 (techo)	30	✓

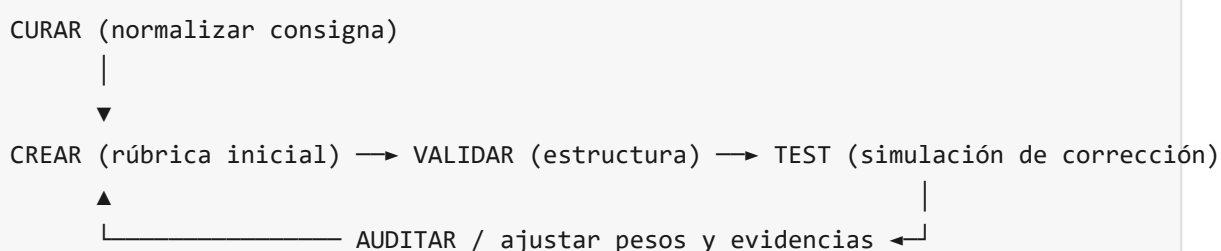
Fuente: [TPs/TP6/sinteticos/2026-07-05_223422/](#).

El caso `s1_falla_c7` arrojó un hallazgo no anticipado: el nombre de campo incorrecto en el nodo Edit Fields (criterio C7) se propagó también al nodo de mensajería (criterio C8), porque su expresión referencia ese mismo campo. La nota obtenida (91) fue más baja que la esperada si solo se penalizara C7 de forma aislada (~95), lo cual demuestra que la rúbrica evalúa correctamente la **coherencia entre nodos del workflow** y no cada nodo de manera independiente — un comportamiento deseable, ya que en un flujo real ese error haría que la automatización fallara en producción. El caso `s2_sin_json_cd2` replica, en el contexto de n8n, el mismo patrón de doble penalización observado en TP1: la falta del archivo `.json` exportado descuenta puntos dentro de C1 y además dispara el techo de `CD2`, que termina siendo el factor decisivo.

9. Ajuste iterativo de consigna, rúbrica y entrega

El proceso descrito en las secciones 4 a 8 no es lineal sino **iterativo**. Si el modo TEST revela que un criterio no discrimina como se esperaba, la causa casi nunca está en el modelo que corrige, sino en el diseño de la rúbrica: una descripción ambigua, una evidencia no verificable, un peso mal distribuido o instrucciones de puntuación poco específicas.

El ciclo de trabajo aplicado en este proyecto fue, en la práctica:



Solo cuando el conjunto de entregas sintéticas produce notas coherentes con lo esperado para cada criterio, la rúbrica se considera lista para cargarse en el sistema real (botón "Cargar criterios" de Active-IA), junto con los campos que no forman parte del JSON y se completan aparte en el formulario (`tipo` , `numero` , `año` , `materia_id`).

10. Caso de estudio aplicado: Organización Empresarial

A lo largo de la cursada de Organización Empresarial se aplicó este flujo de trabajo completo a las rúbricas de los distintos Trabajos Prácticos y del Trabajo Práctico Integrador (TPI), organizados dentro de la carpeta `TPs/` del repositorio:

- **TPs/TP1** — Git y control de versiones (`rubrica-tp1-git.json`)
- **TPs/TP2** — Ingeniería de prompts y pensamiento computacional (`rubrica-tp2-ingenieria-prompts.json`)
- **TPs/TP3** — Metodologías ágiles (`rubrica-tp3-metodologias-agiles.json`)
- **TPs/TP4** — Ecosistema de una startup (`rubrica-tp4-ecosistema-startup.json`)
- **TPs/TP5** — Live coding (`rubrica-tp5-live-coding.json`)
- **TPs/TP6** — N8N (`rubrica-tp6-n8n.json`)
- **TPs/TPI** — Trabajo Práctico Integrador (`rubrica-tpi-oe.json`)

Para cada una de estas rúbricas se generó, además, una carpeta `sinteticos/` con entregas de prueba fechadas y su correspondiente resultado de corrección (`*_correccion.json`) y prompt real enviado al modelo (`*_prompt.txt`), lo que permite trazar de forma completa y auditable el proceso de diseño → testeo → ajuste de cada instrumento de evaluación utilizado en la materia.

Esta decisión metodológica —trabajar siempre con entregas en texto plano y evitar links, imágenes o ejecución de código como requisito de nota— es consistente con el criterio de normalización descripto en la sección 4, y es la razón por la cual estas rúbricas resultaron, en la práctica, altamente corregibles de forma automática y determinista.

11. Conclusiones finales

La implementación de una IA agéntica asistida por una skill especializada demuestra que es posible sistematizar la creación de instrumentos de evaluación —una tarea tradicionalmente artesanal y dependiente del criterio individual del docente— sin perder rigurosidad pedagógica, e incluso ganando en consistencia: cada rúbrica queda validada estructuralmente, curada respecto de los límites técnicos reales del corrector, y testada empíricamente contra entregas de prueba antes de usarse sobre alumnos reales.

El hallazgo conceptual más relevante de todo el proceso es que **la calidad de una corrección automática no depende principalmente del modelo de IA que corrige, sino de la calidad de la rúbrica que se le entrega**. Una rúbrica ambigua, incompleta o con requisitos que dependen de contenido que el corrector estructuralmente no puede leer (links, imágenes, ejecución) producirá correcciones inconsistentes sin importar qué tan avanzado sea el modelo utilizado. Este hallazgo es, precisamente, el motivo por el cual la skill utilizada dedica un modo completo (CURAR) a normalizar la consigna *antes* de siquiera pensar en redactar un criterio.

Como línea de trabajo futura, sería valioso extender este análisis a otras materias con formatos de entrega más heterogéneos, evaluando qué adaptaciones adicionales necesitaría el modo CURAR para esos casos.

12. Fuentes utilizadas y reconocimiento de autoría

⚠ Nota de autoría — de lectura obligatoria

La skill `rubrica-builder` utilizada como herramienta central de este trabajo es **propiedad intelectual de Juan Cruz Robledo**, distribuida bajo licencia **Apache License 2.0** (© 2026 Juan Cruz Robledo), disponible públicamente en el repositorio: <https://github.com/JuanCruzRobledo/rubrica-builder>.

Este Trabajo Final utiliza dicha skill **exclusivamente con fines educativos y de investigación académica**, como caso de estudio para analizar y documentar la aplicación de IA agéntica a la elaboración de rúbricas de evaluación. No se reclama autoría alguna sobre el diseño, la lógica ni el código de la skill en sí misma. Todo el mérito de su concepción, arquitectura y reglas de diseño corresponde a su autor original.

Fuentes y materiales

- Robledo, J. C. *rubrica-builder* (skill para Claude Code). Repositorio oficial: <https://github.com/JuanCruzRobledo/rubrica-builder>. Licencia Apache-2.0.
- Documentación interna de la skill: `SKILL.md`, `README.md`, `references/modelo-rubrica.md`, `references/limites-corrector.md`.
- Anthropic. *Claude Code* — documentación oficial de la IA agéntica utilizada como entorno de ejecución.
- Video propio: "¿Qué es una rúbrica?" — disponible en: drive.google.com/drive/folders/1WewN_uwqNbvnyhkBqQ4WZ2_lge4UHPCU.
- Video propio: "Normalización de la entrega para crear una rúbrica coherente y minimizar fallos" — disponible en: <https://youtu.be/qOXTpZVbJ-s>. Desarrolla el caso de estudio documentado en la sección 4.1: la decisión de estandarizar Git como tecnología de control de versiones y de reemplazar las capturas de pantalla por la carpeta `.git` como evidencia auditable.
- Material propio generado durante la cursada de Organización Empresarial: rúbricas de las Unidades 1 a 6 y del TPI, junto con sus entregas sintéticas de testeo y resultados de corrección, utilizados como caso de estudio aplicado a lo largo de este informe.

— Fin del informe —

